

GPIO Interrupt Controller

Design and Verification Report

A parameterized 8-bit GPIO peripheral with APB control and per-pin interrupt generation, verified using a SystemVerilog class-based testbench.

Status: FUNCTIONALLY VERIFIED

1. Executive summary

The project integrates an APB register interface, GPIO output control, and configurable interrupt detection for eight GPIO pins. Functional verification was performed in XSim. The final environment uses generator, driver, monitor, and scoreboard classes connected by mailboxes.

2. Design blocks

gpio_controller.sv integrates the blocks. gpio_registers.sv stores software-visible controls and produces register reads and interrupt-clear commands. gpio_interrupt.sv detects configured GPIO conditions, maintains sticky interrupt status, and generates IRQ. gpio_io.sv maps output data and direction to GPIO output and output-enable signals.

3. Architecture

APB Master	gpio_controller	gpio_registers	gpio_interrupt	IRQ
			gpio_in + config	
	gpio_io	GPIO pins		

High-level flow: software writes configuration registers -> external GPIO activity is sampled -> interrupt logic detects the configured condition -> status is latched -> enabled status contributes to IRQ -> software clears the pending flag.

4. Register map

Offset	Register	Access	Description
0x00	DATA_IN	RO	Current GPIO input state
0x04	DATA_OUT	R/W	Output value
0x08	DIR	R/W	1=output, 0=input
0x0C	INT_TYPE	R/W	0=level, 1=edge
0x10	INT_POLARITY	R/W	0=low/falling, 1=high/rising
0x14	INT_ENABLE	R/W	Per-pin interrupt enable
0x18	INT_STATUS	RO	Latched pending interrupt flags
0x1C	INT_CLEAR	WO	Write 1 to clear corresponding flag

5. Verification environment

The testbench is intentionally class-based. Generator, driver, monitor, and scoreboard are separate responsibilities, and mailboxes decouple stimulus from checking.

Verification flow

```
gpio_generator -> gen2drv mailbox -> gpio_driver -> DUT -> gpio_monitor -> mon2scb mailbox -> gpio_scoreboard
```

The transaction object also supports GPIO-input stimulus, so the same environment can represent both APB software accesses and an external GPIO event.

6. Final test plan

ID	Stimulus	Expected result
T1	Write DIR = FF	gpio_oe = FF
T2	Read DIR	PRDATA = FF
T3	Write DATA_OUT = A5	gpio_out = A5
T4	Read DATA_OUT	PRDATA = A5
T5	DIR = 00, edge, rising, enable GPIO0	Configuration accepted
T6	gpio_in 00 -> 01	INT_STATUS[0]=1, IRQ=1
T7	INT_CLEAR[0]=1	INT_STATUS[0]=0, IRQ=0

7. Verification result

The final simulation produced:

```
PASS: READ DIR = 0x000000ff
PASS: READ DATA_OUT = 0x000000a5
INFO: Interrupt configured
PASS: INT_STATUS = 1
PASS: IRQ = 1
PASS: INT_STATUS CLEARED
PASS: IRQ CLEARED
```

The final run completed at approximately 370 ns.

8. Key engineering lessons

Separate hardware from verification

The RTL should remain deterministic hardware; the testbench should own stimulus and checking.

Respect simulation scheduling

Nonblocking assignments update in a later scheduling phase; monitors must sample at the right time.

Make end-of-test deterministic

A future version should terminate on a completion event or count rather than relying only on a fixed delay.

Debug from the hierarchy

A waveform becomes much more useful when interface signals and selected DUT internals are shown together.

Do not confuse warnings with failures

Board-part availability and timescale warnings did not prevent successful XSim compilation or simulation.

9. Evidence

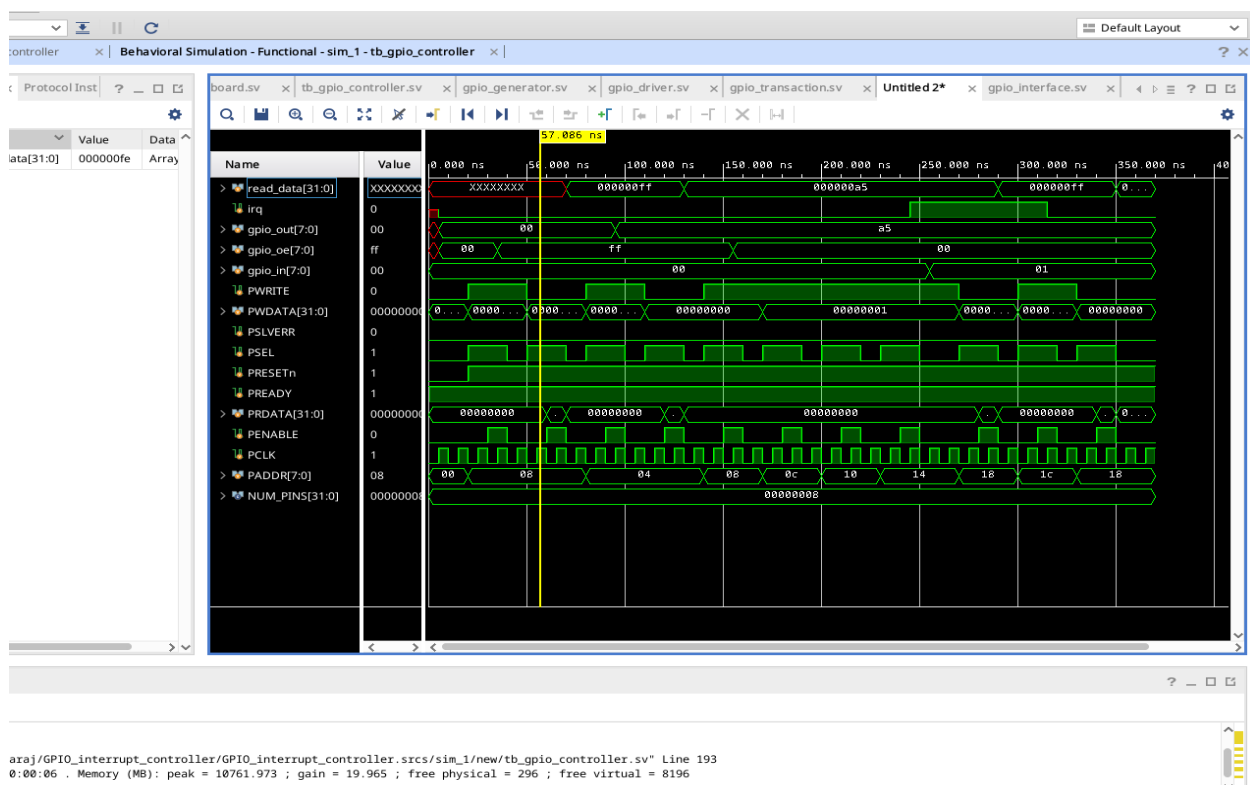


Figure: final waveform evidence.

10. Reproducibility checklist

1. Open the Vivado project.
2. Ensure `tb\_gpio\_controller` is the behavioral simulation top.
3. Confirm all RTL and simulation source files are present.
4. Run Behavioral Simulation.
5. Use Zoom Fit to view the complete timeline.
6. Confirm the seven final PASS/INFO messages in the simulator console.
7. Add APB and GPIO signals to the waveform for inspection.

11. Verification scope and limitations

The current environment verifies the implemented path and the tested interrupt sequence. It is not an exhaustive verification plan. Additional tests for falling edges, level-high/level-low conditions, all eight pins, simultaneous events, invalid addresses, reset behavior, and APB protocol assertions would increase coverage.

12. Repository documentation

The GitHub-ready package accompanying this report contains a professional README, a detailed design/verification document, learning/debug notes, and simulation evidence images. The generated documentation is intentionally explicit about what was tested and what remains outside scope.